

Moduł 3

Refaktoryzacja do wzorca Aggregate

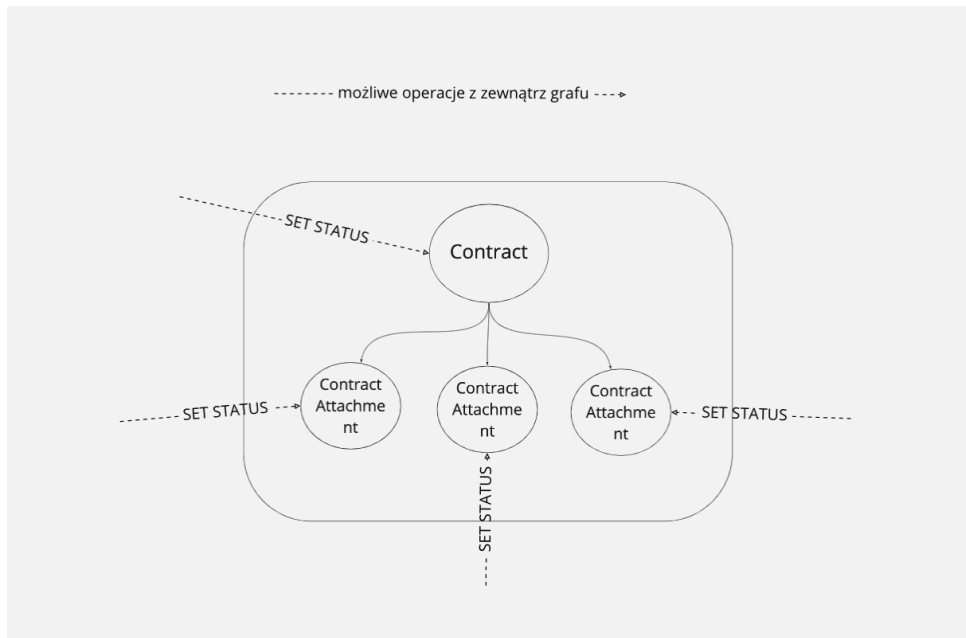
Zadanie domowe - rozwiązanie

Zadanie domowe - rozwiązanie

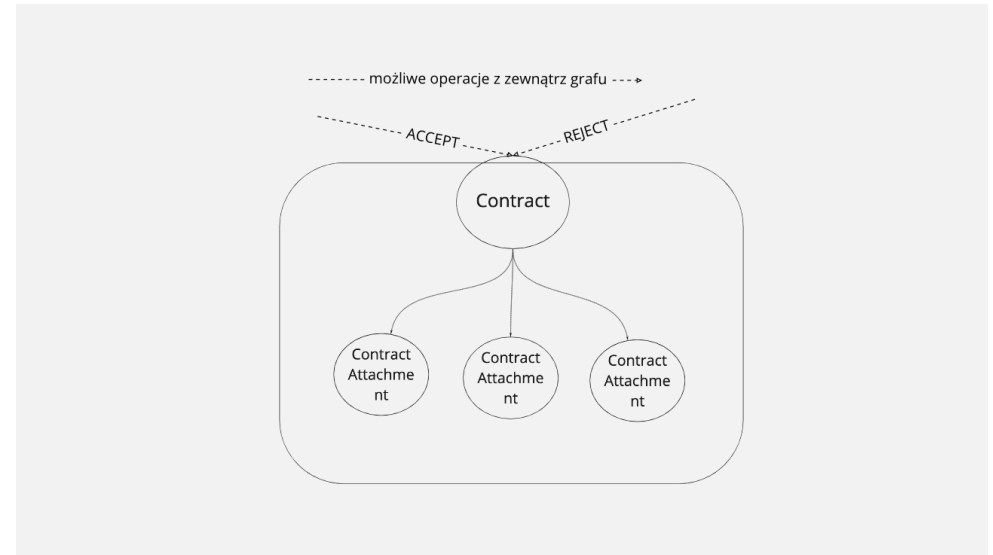
Geneza problemu spójności

Encje `Contract` i `ContractAttachment` to osobne byty, które jednak razem bronią pewnych istotnych reguł. Status kontraktu zależy bezpośrednio od akceptacji (lub jej braku) załączników. Jest więc silna relacja pomiędzy danymi odseparowanymi od siebie w kodzie. Bez większego problemu można ustawić status kontaktu metodą `setStatus(...)`, omijając znajdujące się w serwisach reguły.

Innymi słowy, oba te byty tworzą graf obiektów, które zmieniają się razem i są ze sobą silnie związane. Widać to na Rysunku nr 1. Żeby zachować spójność, należy ten graf zamodelować w kodzie i umożliwić tylko konkretne operacje z zewnątrz. Pokazuje to Rysunek nr 2.



Rysunek 1



Rysunek 2

Obserwowalne zachowania

Obserwowalne zachowania za jakie odpowiadają operacje związane z kontraktami partnerskimi to:

- Tworzenie kontraktu (wraz z odpowiednim i inkrementowanym w ramach jednego partnera numerem `contractNo`)
- Proponowanie załącznika do kontraktu (pojawia się w kontrakcie)
- Usuwanie załącznika z kontraktu (znika z kontraktu)
- Akceptacja załącznika przez jedną stronę (powoduje odpowiednią zmianę statusu)
- Akceptacja załącznika przez obie strony (powoduje odpowiednią zmianę statusu)

- Odrzucenie załącznika przez dowolną ze stron (powoduje odpowiednią zmianę statusu)
- Akceptacja całego kontraktu (możliwa tylko jeśli wszystkie załączniki są zaakceptowane przez obie strony)
- Odrzucenie kontraktu
- Wyświetlanie danych kontraktu wraz z załącznikami

Zabezpieczenie testami

Obserwowalne zachowania można opisać i zabezpieczyć testem integracyjnym na klasie `ContractService`, bo to ona zawiera całą logikę. Nie możemy testować encji `Contract` ani `ContractAttachment`, bo obecnej architekturze są one po prostu strukturami danych. Nie testujemy kontrolera `ContractController`, gdyż ten nie zawiera istotnej logiki.

Test `ContractLifecycleIntegrationTest` znajdziesz w commicie „*Aggregate 3.9 - Contract - zabezpieczenie testami*” i tagu **agg-contract-zabezpieczenie**.

W celach dydaktycznych, test nie zawiera dość trywialnych przypadków typu: „Nie można usunąć nieistniejącego załącznika”.

Wprowadzenie enkapsulacji

Posługując się poznanymi technikami możemy krok po kroku wprowadzać enkapsulację i metody dostępne do encji `Contract`. Poznane techniki to np. Przeniesienie instrukcji warunkowej do encji, szukanie sekwencji tych samych setterów, usunięcie setterów, które są używane tylko przy konstruktorze i przekazanie danych do konstruktora i inne.

Efektom będzie zmniejszenie ryzyka zmiany statusu bez sprawdzenia reguł biznesowych. W idealnym świecie zniknęłoby też repozytorium do załączników (`ContractAttachmentRepository`), a wszystkie zmiany dokonywałyby się z poziomu repozytorium `ContractRepository`, które zarządza całym grafem obiektów. Poniżej widać kilka metod dostępnych, które powstały w wyniku refaktoryzacji klasy `Contract`.

```
public ContractAttachment proposeAttachment(byte[] data) {
    ContractAttachment contractAttachment = new ContractAttachment();
    contractAttachment.setData(data);
    contractAttachment.setContract(this);
    attachments.add(contractAttachment);
    return contractAttachment;
}

public void accept() {
    if (attachments.stream().allMatch(a ->
a.getStatus().equals(ContractAttachment.Status.ACCEPTED_BY_BOTH_SIDES
))) {
        this.status = Contract.Status.ACCEPTED;
    } else {
        throw new IllegalStateException("Not all attachments accepted
by both sides");
    }
}

public void reject() {
    this.status = Status.REJECTED;
}

public void acceptAttachment(Long attachmentId) {
    //..
}
```

```
public void rejectAttachment(Long attachmentId) {
    ContractAttachment contractAttachment =
        findAttachment(attachmentId);
    contractAttachment.setStatus(ContractAttachment.Status.REJECTED);
}
```

Wycinek kodu 1

Zniknęły praktycznie wszystkie settery pozwalająca na wprowadzenie obiektu w dowolny stan (co było wynikowym problemem). A tak wygląda klasa (a właściwie kilka jej metod) ContractService po zmianach:

```
@Transactional
public void acceptContract(Long id) {
    Contract contract = find(id);
    contract.accept();
}

@Transactional
public void rejectContract(Long id) {
    Contract contract = find(id);
    contract.reject();
}
```

Wycinek kodu 2

Instrukcje warunkowe zostały przeniesione do klasy Contract. Dokonane zmiany znajdziesz w commicie: „Aggregate 3.9 - Contract - Zwiększenie enkapsulacji” i tagu **agg-contract-zwiekszenie-enkapsulacji**.

Uwaga: Przy modelowaniu kolekcji jeden do wielu za pomocą narzędzia ORM, najprawdopodobniej kolekcja będzie ładowana w trybie *lazy*. Co oznacza, że ładujemy nasz agregat 2 dwóch różnych momentach czasu - przy

wyciągnięciu kontraktu, a potem przy sprawdzaniu reguły, która dotyczy wszystkich załączników, w metodzie `accept()`. Może to prowadzić do subtelnych niespójności. Rozwiązaniem tego problemu jest ładowanie całego grafu jednocześnie.

Czy Contract ma nowe powstałe testy jednostkowe? Stworzenie ich jest proste, znamy scenariusze, jest to podzbiór wszystkich testów integracyjnych wykonanych w pierwszym commicie. Czy utworzyliśmy te testy? Jeszcze nie, bo nie rozwiązaliśmy drugiego problemu. Tego, że akceptacja kontraktu czasem zawiesza system...

Geneza drugiego problemu

Powstały obiekt nie jest duży. Nie jest duży statycznie - ma tylko kilka pól, kilka danych. Jednym z pól jest kolekcja załączników `ContractAttachment`. Mimo, że rozmiar obiektu nie przeszkadza nam pod względem czytelności czy też testowalności, jego rozmiar może przeszkadzać w tzw. „run-time”. Dlaczego? `ContractAttachment` ma pole `data` o typie `byte[]`. Oznacza to, że w przypadku rozmiaru tego pola w bazie danych (np. duży plik *avi* czy *pdf*) aby zaakceptować kontrakt musimy zaciągnąć go do pamięci aplikacji.

Czy jest jakakolwiek reguła, która pozwala na akceptację kontraktu wtedy i tylko wtedy gdy załącznik ma konkretne dane? Nie, wystarczy nam status akceptacji tego i innych załączników. Co oznacza, że granice naszego agregatu są zbyt duże, wciąga on dane, które najpewniej potrzebne są tylko w celach odczytowych, a nie w celach podjęcia decyzji.

Uwaga: Rozmiar obiektu nie musi być duży pod względem struktury kodu. Może być duży pod względem pamięci w aplikacji.

Rozbicie obiektu

Rozwiązaniem jest wyciągnięcie problematycznej danej `byte[] data` do innego obiektu. Ten algorytm już znamy - należy rozbić obiekt i zmigrować dane. Wycinek numer 3 pokazuje nową stworzoną strukturę:

```
@Entity
public class ContractAttachmentData {

    @Column(nullable = false)
    private UUID contractAttachmentNo;

    @Lob
    @Column(name = "data", columnDefinition = "BLOB")
    private byte[] data;

    @Column(nullable = false)
    private Instant creationDate = Instant.now();
    //...
}
```

Wycinek kodu 3

Co oznacza, że "odchudzona" encja `ContractAttachment` wygląda teraz tak:

```
@Entity
public class ContractAttachmentData {

    @Column(nullable = false)
    private UUID contractAttachmentNo = UUID.randomUUID();
```

```
@Lob
@Column(name = "data", columnDefinition = "BLOB")
private byte[] data;

private Instant acceptedAt;

private Instant rejectedAt;

private Instant changeDate;
//...
}
```

Wycinek kodu 4

Warto wspomnieć o dwóch zmianach:

- 1) W obu encjach pojawiło się pole `UUID contractAttachmentNo`. Będzie to dana, która mapuje `ContractAttachment` do `ContractAttachmentData` i na odwrót. Nie używamy do tego bazodanowego id, bo to powstaje dopiero po zapisie. Nie mapujemy też jednej encji do drugiej poprzez referencję, żeby nie było możliwości zmiany `ContractAttachment` NIE poprzez agregat `Contract`.
- 2) Do `ContractAttachmentData` przeniesiono też datę `creationDate`. Dlaczego? Dlaczego w `ContractAttachment` zostały daty `acceptedAt`, `rejectedAt` i `changeDate`? Jakim kluczem wyboru się tu kierowano? Data `creationDate` oraz tablica danych `data` to dane które nigdy się nie zmieniają i nie są potrzebne do reguł w agregacie `Contract`. Stąd są na zewnątrz. Ale pozostałe trzy daty też nie są potrzebne do reguł, dlaczego więc nie zostały usunięte? Dlaczego w `ContractAttachment` nie został praktycznie tylko status, który do reguł jest potrzebny? Odpowiedź jest prosta: `Contract` zarządza też cyklem spójności i to on ma metody `rejectAttachment()` i `acceptAttachment()` (Wycinek 2),

które wewnątrz zmieniają statusu i potencjalnie też datę. Agregat zapewnia tutaj spójność danych, jest w końcu jednostką transakcji - obie dane zostaną w tej transakcji ustawione. To dlaczego nie są? Bo przed refaktoryzacją nie były, być może jest to błąd w systemie. W każdym razie, budowa proponowanego obiektu jest gotowa, aby ten ewentualny błąd naprawić.

Skoro tylko Contract ma "pozwolenie" na modyfikację ContractAttachment to możemy i powinniśmy pozbyć się repozytorium ContractAttachmentRepository. Wszystkie zmiany będą wykonywane poprzez wczytywanie i zapisywanie całego obiektu Contract. Pojawi się za to ContractAttachmentDataRepository. Zapiszemy przez to repozytorium daną, która powodowała problemy z pamięcią.

Wczytywanie Contract

Wspomnieliśmy o problemie wczytywania do pamięci aplikacji encji Contract w 2 „porcjach”. Aby tego uniknąć można skonfigurować ORM, aby wczytywał Contract wraz zależnymi ContractAttachment za pomocą jednej instrukcji typu JOIN. Można to zaobserwować poniżej:

```
@OneToMany(mappedBy = "contract", cascade = CascadeType.ALL)
@Fetch(value = FetchMode.JOIN)
private Set<ContractAttachment> attachments = new HashSet<>();
```

Wycinek kodu 5

Alternatywą byłoby przeniesienie wszystkich tych związanych z załącznikami danych do tabeli contract w bazie danych. Mogłaby to być kolumna typu BLOB. W takim wypadku - nasz agregat składałby się z 1 tabeli.

Budowanie widoku

Dotychczas klasa Contract miała wszystko, żeby zbudować z niej widok, z użyciem ContractDTO. Po odchudzeniu obiektu Contract, a właściwie ContractAttachment, konstruktor ContractDTO musi wyglądać następująco:

```
public ContractDTO(Contract contract, Set<ContractAttachmentData>
attachments) {
```

Wycinek kodu 6

Zauważ, że z punktu widzenia obserwowalnych zachowań nie ma znaczenia JAK ten widok budujemy. Ważne jest, że ma te same dane. Dlatego napisane na samym początku testy pozostają niezmienione i są zielone.

Testy jednostkowe

Agregat Contract może być teraz przetestowany na poziomie swoich metod. Te metody przejmą niektóre obserwowalne zachowania testowane wcześniej (przed refaktoryzacją) na poziomie ContractService. Możemy pozbyć się tej duplikacji. Poniżej widać przykładowy test jednostkowy. Wszystkie testy znajdziesz w klasie io.legacyfighter.cabs.entity.ContractLifecycleTest. Commit rozbijający obiekty jest podpisany „Aggregate 3.9 - Contract - Rozbicie obiektów” i tagu **agg-contract-rozbicie**.

```
@Test
void canAcceptContractWhenAllAttachmentsAccepted() {
    //given
```

```
Contract contract = createContract("partnerName", "umowa o cenę");
//and
ContractAttachment attachment = contract.proposeAttachment();
//and
contract.acceptAttachment(attachment.getContractAttachmentNo());
contract.acceptAttachment(attachment.getContractAttachmentNo());

//when
contract.accept();

//then
assertEquals(Contract.Status.ACCEPTED, contract.getStatus());
}
```

Wycinek kodu 7

Nazewnictwo i migracja danych

Czy ContractAttachment to na pewno jest dobra nazwa po takiej refaktoryzacji? Można się nad tym zastanowić. Ta kolekcja encji zawiera tylko decyzję o akceptacji (lub jej braku) jakiegoś załącznika. Nie zawiera danych o samym załączniku. Być może przy skrypcie migrującym dane z tabelki contract_attachment do contract_attachment_date można zastanowić się nad inną nazwą tabeli i klasy. ContractAttachmentDecision wydaje się bardziej pasować do tak zamodelowanego grafu obiektów.